

Connect Four — Vibe Coding Chat History

Tasks 1–4 | Prompts and Responses

Task 1 — User Prompt

****Task 1 — Complete win detection****

Review my current `board.py` for the Connect Four project and complete Task 1 from the README:

“Make win detection recognise every four-in-a-row direction, including both diagonals, without breaking horizontal and vertical wins.”

The current `winner()` method should detect horizontal, vertical, and both diagonal four-in-a-row patterns. Also ensure sequences shorter than four do not count as a win.

Do not rewrite unrelated parts of the project. Keep the existing `Board` class and method interface. Explain the change before providing the code.

```
board.py
from board import Board
from ai import AI

class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]

    def drop(self, col, token):
        if not 0 <= col < COLS:
            return None
        for r in range(ROWS - 1, -1, -1):
            if self.grid[r][col] == ".":
                self.grid[r][col] = token
                return r
        return None

    def full(self):
        return all(self.grid[0][c] != "." for c in range(COLS))

    def winner(self, token):
        directions = [(0, 1), (1, 0)]
        for r in range(ROWS):
            for c in range(COLS):
                if self.grid[r][c] != token:
                    continue
                for dr, dc in directions:
                    if all(
                        0 <= r + i * dr < ROWS and
                        0 <= c + i * dc < COLS and
                        self.grid[r + i * dr][c + i * dc] == token
                        for i in range(4)
                    ):
                        return True
        return False

    def print(self):
        print("\n " + " ".join(str(i + 1) for i in range(COLS)))
        for row in self.grid:
            print(" " + " ".join(row))
```

Task 1 — LLM Response (ChatGPT)

Your current `winner()` already checks `**horizontal**` `(0, 1)` and `**vertical**` `(1, 0)` wins.

The only missing part is the `**two diagonal directions**`:

- `■` down-right: `(1, 1)`
- `■` down-left: `(1, -1)`

We can simply add those directions to the existing list. The ``all(... for i in range(4))`` already ensures that `**exactly four consecutive matching positions are required**`, so sequences of 1–3 will not count.

No other part of ``Board`` needs to change.

```
class Board:
    def __init__(self):
        self.grid = [ "." * COLS for _ in range(ROWS) ]

    def drop(self, col, token):
        if not 0 <= col < COLS:
            return None
        for r in range(ROWS - 1, -1, -1):
            if self.grid[r][col] == ".":
                self.grid[r][col] = token
                return r
        return None

    def full(self):
        return all(self.grid[0][c] != "." for c in range(COLS))

    def winner(self, token):
        # Horizontal, vertical, diagonal down-right, diagonal down-left
        directions = [
            (0, 1), # horizontal →
            (1, 0), # vertical ↓
            (1, 1), # diagonal ■
            (1, -1) # diagonal ■
        ]

        for r in range(ROWS):
            for c in range(COLS):
                if self.grid[r][c] != token:
                    continue

                for dr, dc in directions:
                    if all(
                        0 <= r + i * dr < ROWS
                        and 0 <= c + i * dc < COLS
                        and self.grid[r + i * dr][c + i * dc] == token
                        for i in range(4)
                    ):
                        return True

        return False

    def print(self):
        print("\n " + " ".join(str(i + 1) for i in range(COLS)))
        for row in self.grid:
            print(" " + " ".join(row))
```

What changed

Only this:

```
directions = [
    (0, 1),
    (1, 0),
    (1, 1),
    (1, -1)
]
```

So `winner()` now checks all four possible Connect Four directions:

```
→ horizontal
↓ vertical
■ diagonal
■ diagonal
```

And because it uses:

```
for i in range(4)

**3 in a row → no win**
**4 in a row → win**
**5+ in a row → also win** (because it contains a four-in-a-row).
```

This completes **Task 1** without changing the existing `Board`` interface or unrelated functionality.

Task 2 — User Prompt

Task 2 — Complete game termination

Review my current `game.py`` for the Connect Four project and complete Task 2 from the README:

“Handle draws, full columns, invalid moves, and game termination consistently. A winning move must stop the game before another turn is requested.”

Requirements:

- Invalid input must not change the board.
- Full columns must not change the board.
- Draws must be detected when the board is full.
- A winning move must immediately end the game.
- The game must not request another turn after a win or draw.
- Handle an unavailable AI move safely.

Make the minimum necessary changes to the existing `game.py``. Do not rewrite unrelated code or change other files.

Explain the changes before providing the complete modified `game.py``.

```
game.py
from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four — you are X.")
        while True:
            self.board.print()
            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()
                if raw == "q":
                    return
                try:
                    col = int(raw) - 1
                except ValueError:
                    print("Enter a column number.")
                    continue
                else:
                    col = self.ai.choose_column(self.board)

            if col is None or self.board.drop(col, self.turn) is None:
                print("Column unavailable.")
            if self.turn == "O":
                return
            continue
```

```

if self.board.winner(self.turn):
    self.board.print()
    print(self.turn, "wins!")
    return
if self.board.full():
    self.board.print()
    print("Draw.")
    return

self.turn = "O" if self.turn == "X" else "X"

```

Task 2 — LLM Response (ChatGPT)

The current `game.py` is already close. The main issue is that **AI move failure is treated as a generic unavailable column and immediately exits**, while the requirements call for safely handling an unavailable AI move.

Changes made

1. **Invalid human input**
 - Non-numeric input still leaves the board unchanged.
 - `q` exits normally.
 - Out-of-range numbers such as `0` or `8` are rejected before calling `drop()`.
2. **Full columns**
 - `board.drop()` returns `None`, so the board is unchanged.
 - For the human player, the game asks for another valid move.
 - For the AI, the game safely ends if it cannot make a move.
3. **Win before turn change**
 - After a successful move, `winner()` is checked immediately.
 - The function returns immediately, so **no next turn is requested**.
4. **Draw**
 - `full()` is checked after the win check.
 - If the board is full, the game ends immediately.
5. **Unavailable AI move**
 - If `choose\_column()` returns `None`, or the selected column cannot accept a piece, the game terminates safely rather than trying to continue with an invalid turn.

Only `game.py` is changed.

```

from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four – you are X.")

        while True:
            self.board.print()

            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()

                if raw == "q":
                    return

            try:
                col = int(raw) - 1
            except ValueError:
                print("Enter a column number.")
                continue

```

```

        if not 0 <= col < 7:
            print("Enter a column number.")
            continue

    else:
        col = self.ai.choose_column(self.board)

        # AI could not find a valid move
        if col is None:
            self.board.print()
            print("AI has no available move. Draw.")
            return

        # Move is invalid or column is full
        if self.board.drop(col, self.turn) is None:
            print("Column unavailable.")

            if self.turn == "O":
                return

            continue

        # Check win immediately after the move
        if self.board.winner(self.turn):
            self.board.print()
            print(self.turn, "wins!")
            return

        # Check draw before requesting another turn
        if self.board.full():
            self.board.print()
            print("Draw.")
            return

    self.turn = "O" if self.turn == "X" else "X"

```

One important detail

I used:

```
if not 0 <= col < 7:
```

before `drop()` for human input. This guarantees inputs like ``0``, ``8``, ``-1``, etc. ****cannot modify the board****.

The termination order is now:

```

Make valid move
↓
Check winner
↓
Win? → STOP
↓
Check board full
↓
Draw? → STOP
↓
Switch turn
↓
Request next turn

```

So a winning move ****always stops the game before another turn is requested****, satisfying the key Task 2 requirement.

Task 3 — User Prompt

****Task 3 — Improve the Connect Four AI****

Review my current ``ai.py`` and complete Task 3 from the README.

The AI must:

1. Take an immediate winning move if one exists.
2. Block the opponent if they have an immediate winning move.
3. Never select a full or illegal column.
4. Return ``None`` safely if no legal columns are available.

Keep the existing `AI.choose_column(self, board, me="O", opponent="X")` interface and do not rewrite unrelated files.

You may use the existing `Board.winner()` and `Board.drop()` methods. If you temporarily modify the board to test a possible move, make sure the board is restored exactly afterward.

Explain the changes before giving the code.

```
ai.py
import random

class AI:
    def choose_column(self, board, me="O", opponent="X"):
        # Starter AI is deliberately basic; students improve it in Task 3.
        legal = [c for c in range(7) if board.grid[0][c] == "."]
        return random.choice(legal) if legal else None
```

Task 3 — LLM Response (ChatGPT)

The current AI only chooses a random legal column. For Task 3, we can keep the implementation small and use `Board.drop()` + `Board.winner()` to test moves.

Changes

The AI will check columns in this order:

1. **Winning move:** temporarily place `me` and check whether it wins.
2. **Blocking move:** temporarily place `opponent` and check whether that would win, then choose that column to block.
3. **Otherwise:** choose a random legal column.
4. **No legal moves:** return `None`.

For every temporary move, the original cell is restored to `."`, so the board is unchanged after testing.

```
import random

class AI:
    def choose_column(self, board, me="O", opponent="X"):
        legal = [c for c in range(7) if board.grid[0][c] == "."]

        if not legal:
            return None

        # 1. Take an immediate winning move
        for col in legal:
            row = board.drop(col, me)

            if row is not None:
                won = board.winner(me)
                board.grid[row][col] = "."

                if won:
                    return col

        # 2. Block the opponent's immediate winning move
        for col in legal:
            row = board.drop(col, opponent)

            if row is not None:
                won = board.winner(opponent)
                board.grid[row][col] = "."

                if won:
                    return col

        # 3. Otherwise choose any legal column
        return random.choice(legal)
```

Why this satisfies Task 3

Winning move:

```
row = board.drop(col, me)
if board.winner(me):
    return col
```

The AI tests whether placing `O` in that column creates four-in-a-row.

****Blocking:****

```
row = board.drop(col, opponent)
if board.winner(opponent):
    return col
```

This asks: **"If X played here, would X win?"** If yes, the AI chooses that same column to prevent it.

****Board restoration:****

```
board.grid[row][col] = "."
```

Every temporary move is removed immediately after testing, so these simulations don't affect the actual game.

****Full/illegal columns:****

```
legal = [c for c in range(7) if board.grid[0][c] == "."]
```

Only columns with space at the top are considered.

****No available moves:****

```
if not legal:
    return None
```

This also works with your updated `game.py`, which safely handles an AI returning `None`.

Task 4 — User Prompt

****Task 4 — Add move-level feedback****

Review the current Connect Four project and complete Task 4 from the README.

Add concise feedback when a disc is successfully placed. The feedback must happen exactly once per actual move.

It must NOT be printed once for every cell inspected during `winner()` or AI analysis.

Keep the existing game structure and make the minimum necessary changes. Do not add persistent storage or unrelated dependencies.

Explain the change and provide the modified code.

README.md

Scenario 15 — Connect Four vs AI

A terminal Connect Four game with a board module, a player turn loop, and a simple AI.

Provided files

- `main.py` — entry point.
- `game.py` — turn handling and game flow.
- `board.py` — board state, drops, and win detection.
- `ai.py` — computer move selection.
- `requirements.txt` — dependency declaration.

Setup

```
python main.py
```

Before changing the code

Play several games, inspect all modules, and construct horizontal, vertical, and diagonal winning positions. Trace how a move travels from the command line to the board and then to winner detection.

Task 1 — Complete win detection

Make win detection recognise every four-in-a-row direction, including both diagonals, without breaking horizontal and vertical wins.

****Done when:**** all four directions are detected and shorter sequences do not count.

Task 2 — Complete game termination

Handle draws, full columns, invalid moves, and game termination consistently. A winning move must stop the game before another turn is requested.

****Done when:**** no invalid move changes the board and every terminal state is clear.

Task 3 — Improve the AI

Add meaningful decision-making. The AI should take an immediate winning move when one exists and block an immediate player win when necessary. Handle full columns safely.

****Done when:**** the AI never selects an illegal column and responds correctly to one-move threats.

Task 4 — Move-level feedback

Add concise feedback for a successful disc placement. It should occur once per actual move, not once per cell inspected by win detection or AI analysis.

Required testing

Test horizontal, vertical, both diagonals, draw positions, full columns, invalid input, immediate AI wins, immediate AI blocks, and quitting.

LLM usage

You may use an LLM during the lab. The goal is to use it as a coding assistant while retaining responsibility for understanding and testing the result.

- Inspect the existing code before asking for changes.
- Ask for explanations when you do not understand a proposed change.
- Test generated code against the stated behaviour and edge cases.
- Keep your complete LLM chat history for submission.
- Do not replace the whole project with an unrelated implementation.
- Keep all state in memory; do not add CSV, JSON, SQLite, or other persistence.

Submission checklist

- [] Task 1 completed and the original defect was reproduced and fixed.
- [] Tasks 2–4 completed and tested.
- [] Boundary and invalid-input cases tested.
- [] No unnecessary external dependencies added.
- [] No persistent storage added.
- [] Code remains understandable and modular.
- [] Complete LLM chat-history link included.

Folder structure

```
scenario-03-connect-four/  
■■■ README.md  
■■■ requirements.txt  
■■■ main.py  
■■■ game.py  
■■■ board.py  
■■■ ai.py
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay ****before**** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay ****after**** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

Task 4 — LLM Response (ChatGPT)

For Task 4, the feedback should be attached to the **actual successful `drop()` operation**, not to `winner()` or `AI.choose_column()`.

The cleanest minimum change is in `game.py`, immediately after:

```
self.board.drop(col, self.turn)
```

succeeds. This guarantees:

- **1 message per actual move**
- Invalid moves → **no feedback**
- Full columns → **no feedback**
- AI simulations → **no feedback**
- `winner()` checking cells → **no feedback**
- The existing `Board` and `AI` logic stays unchanged

For example, after a successful move, it can print:

```
X placed in column 4.
```

Modified `game.py`

```
from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four – you are X.")

        while True:
            self.board.print()

            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()

                if raw == "q":
                    return

                try:
                    col = int(raw) - 1
                except ValueError:
                    print("Enter a column number.")
                    continue

                if not 0 <= col < 7:
                    print("Enter a column number.")
                    continue

            else:
                col = self.ai.choose_column(self.board)

                if col is None:
                    self.board.print()
                    print("AI has no available move. Draw.")
                    return

            # Attempt the actual move
            row = self.board.drop(col, self.turn)

            if row is None:
                print("Column unavailable.")

                if self.turn == "O":
                    return

            continue

            # Task 4: feedback exactly once for each successful move
            print(self.turn, "placed in column", col + 1)
```

```

# Check win immediately after the actual move
if self.board.winner(self.turn):
    self.board.print()
    print(self.turn, "wins!")
    return

# Check draw before requesting another turn
if self.board.full():
    self.board.print()
    print("Draw.")
    return

self.turn = "O" if self.turn == "X" else "X"

```

Why this is the right location

The important change is:

```

row = self.board.drop(col, self.turn)

if row is None:
    ...
    continue

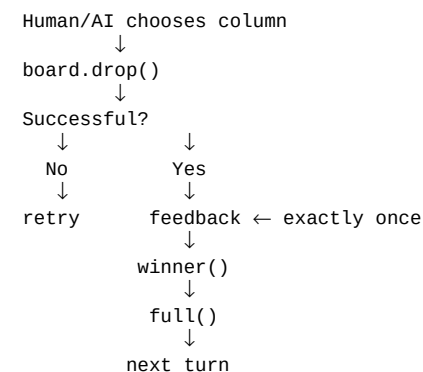
print(self.turn, "placed in column", col + 1)

```

`drop()` returning a row means a disc was **actually placed**. Therefore the feedback cannot occur for an invalid or full-column attempt.

Also, your Task 3 AI temporarily calls `board.drop()` while testing possible moves, but **that happens inside `ai.py`**, where no feedback is printed. The AI then restores the cell.

So the feedback happens only at the real game-level move:



This completes **Task 4** without changing `board.py`, `ai.py`, or the overall game structure.